

Episode 02: Unknown Systematic Errors

Or: Why All Models Are Wrong, But Some Are Useful

Manfred Wiessner

September 22, 2026

1 Introduction

Speaker Script & Visualization

[00:00–01:30] **Welcome & Problem Statement**

Camera: Host directly to camera, calm framing, optional overlay of the episode title as text overlay.

Speaker: Welcome to Episode 2! Today, we want to dedicate ourselves to a topic that is all too often downplayed or swept under the rug in practice: How do we deal meaningfully with *unknown* systematic errors?

This topic is huge, and unfortunately I cannot cover all available methods in this single episode. I ask for a little patience here—if there is interest in a specific subtopic, I would be happy to make a dedicated video about it!

Before we dive deep, however, we must first ask ourselves a fundamental question: How do systematic errors arise in the first place?

2 How Do Unknown Systematic Errors Arise?

Speaker Script & Visualization

[01:30–04:30] **Model Gaps, Conditioning & ELS Framework**

Camera: Host on screen. When quoting George Box, display text graphic. Later, schematic representation: Model vs. Reality (Residu-

als).

Speaker: Statistician George Box once put it perfectly: “*All models are wrong, but some are useful.*”

Why is that? Every physical or mathematical model has its limitations. It always describes only a simplified part of complex reality. Precisely these discrepancies between our simplified model and the actual data are the *unknown systematic errors*.

Now, one could try to make the model arbitrarily complex to capture reality more accurately. But unfortunately, that doesn’t help at all! Complex models require increasingly more model parameters. When analyzing measurement data, this quickly leads to *ill-conditioned systems of equations*.

The consequence: The explanatory power of the parameters shrinks drastically because many completely different parameter combinations suddenly describe the problem equally well.

It was precisely for this reason that we developed and published an *entire framework* addressing this problem. Since our code is fully available as open source, you are welcome to use our concept for your own analyses and projects. Of course, we would be thrilled if you cited our work!

3 How Can We Describe This? (The Bayesian Approach)

Speaker Script & Visualization

[04:30–08:00] Bayesian Modeling, Nuisance Parameters & Terminology

Camera: Close-up on host. When mentioning technical terms (Bayes, MCMC, marginalization), subtly show term cards at the screen edge. Later, schematic diagram: Broadening of the posterior distribution.

Speaker: If we want to capture this unknown systematic error mathematically, there is an extremely elegant path from Bayesian statistics: We introduce so-called **nuisance parameters**. But as you will see shortly, our own new ELS approach from the paper is even more powerful—because we explicitly model the residuals and their standard deviations directly.

First, let's look at the classic Bayesian approach: Instead of pretending we don't know the exact form of the bias, we add an extra parameter δ to our model representing this unknown drift or offset. Since we don't know the exact magnitude of δ before measurement, we assign it a **prior**—for instance, a normal distribution centered at 0 with a realistic standard deviation.

Now I realize: That's a lot of heavy technical terminology all at once! To be honest, there are three distinct major topics here that I cannot cover in detail in this video:

- **Bayesian Statistics:** A fundamentally alternative perspective to classical (frequentist) statistics. Here, parameters are treated as probability distributions rather than fixed, unknown constants.
- **Markov Chain Monte Carlo (MCMC):** A powerful numerical technique to simulate stochastic chains and generate an exact representation of high-dimensional distributions.
- **Marginalization:** The mathematical procedure where we integrate (or sum) over all possible states of our nuisance parameter δ to remove it from the final result.

Thank you for understanding that we cannot go down every rabbit hole here. However, if you would like a dedicated introductory video on Bayesian statistics or MCMC sampling, let me know in the comments—I'd be glad to create a separate video for it!

So, what is the crucial advantage of this Bayesian approach?

Through the marginalization of the nuisance parameter δ , the bias disappears from our primary focus, but its uncertainty remains preserved! The mathematical result is fascinating: **Our credible intervals (posterior intervals) automatically widen.**

We are no longer fooling ourselves with overly narrow error bars; instead, the model reflects the honest, expanded uncertainty caused by the unknown systematic error.

4 And What If I Want a Reliable Error Estimation? – Extended Least Squares

Speaker Script & Visualization

[08:00–12:00] **Our Framework: Simultaneous Parameter & Error Modeling**

Camera: Host. As formulas are mentioned, switch to screen share / overlay showing the mathematical derivation from your publication.

Speaker: Now we come to the core and the solution we developed and published: What do we do if we want to determine not only the model parameters, but simultaneously require an honest, mathematically sound **residual estimation** for our measurement data?

The key is: We simply model the uncertainty—that is, the standard deviations—of the residuals directly! Alongside the pure model parameters $\vec{\xi}_1$, we introduce a second set of parameters $\vec{\xi}_2$ describing the behavior of the residual standard deviations—our so-called **error model** $s_{c,i}(y_{c,i}(\vec{\xi}_1), \vec{\xi}_2)$.

Starting from a Gaussian likelihood function in Bayesian statistics, we transform the problem of maximizing the log-posterior distribution into minimizing an extended objective function Z . By multiplying the log-posterior by (-2) , we bring the problem into a form strongly resembling the classical least-squares sum:

$$\begin{aligned} Z &= -2 \left[\ln P \left(\{\vec{\xi}_1, \vec{\xi}_2\} \mid \{(x_i, y_i)\}, M \right) \right. \\ &\quad \left. + \ln P \left(\{(x_i, y_i)\} \mid M \right) + \frac{n}{2} \ln(2\pi) \right] \\ &= \sum_{i=1}^n \left\{ 2 \ln \left[s_{c,i} \left(y_{c,i}(\vec{\xi}_1), \vec{\xi}_2 \right) \right] + \frac{\left[y_i - y_{c,i}(\vec{\xi}_1) \right]^2}{s_{c,i}^2 \left(y_{c,i}(\vec{\xi}_1), \vec{\xi}_2 \right)} \right\} \\ &\quad - 2 \ln P \left(\{\vec{\xi}_1, \vec{\xi}_2\} \mid M \right) \end{aligned}$$

Assuming a flat, non-informative prior ($P(\vec{\xi} \mid M) = 1$), the log-prior term becomes zero, leaving our simplified objective function Z_1 :

$$Z_1 = \sum_{i=1}^n \left\{ 2 \ln \left[s_{c,i} \left(y_{c,i}(\vec{\xi}_1), \vec{\xi}_2 \right) \right] + \frac{\left[y_i - y_{c,i}(\vec{\xi}_1) \right]^2}{s_{c,i}^2 \left(y_{c,i}(\vec{\xi}_1), \vec{\xi}_2 \right)} \right\}$$

Let me draw your attention to equation Z_1 in detail!

The second term in the sum is the standard weighted sum of squared residuals $\frac{(y_i - y_{c,i})^2}{s^2}$. If we were to minimize only this term, the algorithm would try to make s infinitely large to push the deviation to zero—causing the system to collapse.

The first term prevents this: $2 \ln(s_{c,i})$. This logarithmic term acts as the mathematical penalty for excessively large uncertainties!

By minimizing Z_1 , we perform an **Extended Least Squares (ELS)** fit, which is equivalent to Maximum Likelihood Estimation and the MAP (*Maximum A Posteriori*) solution.

The huge practical advantage: Not only do we obtain robust parameter estimates $\vec{\xi}_1$, but through the **Hessian matrix** of the objective function, we directly get the complete covariance matrix—including all error bounds, parameter correlations, and credible intervals for our predictions.

How this is implemented in Python—for example, when determining dislocation densities in materials science or analyzing spectra—we will look at in the code now!

5 Practical Example: Microstructural Evolution During Stainless Steel Tempering

Speaker Script & Visualization

[12:00–16:30] **Application Example: Dislocation Density in Chromium Steel (N404)**

Camera: Host. When mentioning measurements, display text overlays showing the alloy and Boltzmann function. Later, switch to plots and tables.

Speaker: Let's look at a concrete real-world experiment from materials science! We examine the temperature-dependent evolution of the dislocation density ρ in martensite during tempering of chromium stainless steel—grade Böhler N404.

Dislocation density strongly influences mechanical properties like tensile strength and toughness. From in-situ X-ray diffraction data, the reduction in dislocations across temperature was determined. Physically, this process can be described by thermally activated recovery processes showing a characteristic sigmoidal curve.

As a phenomenological model, we use a **Boltzmann function**:

$$\rho(\vartheta) = \rho_2 + \frac{\rho_1 - \rho_2}{1 + \exp\left(\frac{\vartheta - \vartheta_0}{\Delta\vartheta}\right)}$$

Here, ρ_1 and ρ_2 are the initial and final dislocation densities, ϑ_0 is the transition midpoint temperature, and $\Delta\vartheta$ is the transition temperature interval.

Now comes the crucial question for our ELS framework: **How do we choose the error function s_c ?**

When measured values span several orders of magnitude (from 10^{13} to 10^{15} m^{-2}), a constant uncertainty $s_c = s_0$ or a square-root dependence $s_1\sqrt{\rho}$ often results in mathematical failure at high temperatures: The confidence or credible interval drops into negative values! However, negative dislocation densities are physically impossible.

The solution: We select a **linear error model** that scales with density:

$$s_c(\vartheta) = s_0 + s_1 \cdot \rho(\vartheta)$$

Using our ELS approach, we now estimate **all 6 parameters simultaneously**—the 4 physical parameters of the Boltzmann function as

well as the 2 parameters s_0 and s_1 of our error model!

Looking at the results:

- We obtain precise estimates such as $\vartheta_0 = 543.2 \pm 1.4^\circ\text{C}$ and $\Delta\vartheta = 19.2 \pm 1.3\text{ K}$.
- Via the ****Hessian matrix**** of Z_1 , we compute the full correlation matrix $\overleftrightarrow{C}$.
- We observe an anti-correlation of -0.75 between $\Delta\vartheta$ and ρ_2 , and -0.92 between error parameters s_0 and s_1 .

The result is a physically consistent uncertainty band ($\pm 1s_c$) that remains strictly positive at all temperatures and captures experimental scatter accurately!

6 Complete Python Script

The following script performs the Extended Least Squares fit for the degradation of dislocation density in stainless steel (N404), calculates the Hessian matrix and parameter correlations, and visualizes the fit alongside uncertainty bands and residuals.

Listing 1: Complete source code for ELS analysis and uncertainty estimation

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from scipy.optimize import minimize
import numdifftools as nd

#
# 1. MEASUREMENT DATA AND INITIAL PARAMETERS
#
x_data = np.array([
    22, 22, 27.5, 42.5, 50, 50, 62.5, 87.5, 100, 100,
    112.5, 137.5,
    150, 150, 162.5, 187.5, 200, 200, 212.5, 237.5,
    250, 250, 262.5,
    287.5, 300, 300, 312.5, 337.5, 350, 350, 362.5,
    387.5, 400, 400,
    412.5, 437.5, 450, 450, 462.5, 487.5, 500, 500,
    512.5, 537.5, 550,
    550, 562.5, 587.5, 600, 600, 612.5, 637.5, 650
])

# Scaling y-data by factor 100 for numerical stability
scaling_factor = 0.01
y_data_original = np.array([
    780.736, 694.997, 634.522, 710.078, 652.855,
    732.355, 732.608,
    660.579, 640.615, 706.121, 721.219, 729.574,
    701.986, 721.649,
    629.576, 641.398, 660.333, 660.576, 647.118,
    672.643, 716.937,
```

```

        646.079, 635.233, 618.923, 620.085, 621.435,
        609.958, 630.906,
        634.038, 621.224, 647.268, 621.815, 644.310,
        699.290, 628.957,
        642.193, 624.068, 610.783, 607.593, 605.209,
        578.407, 517.969,
        570.978, 513.906, 291.448, 216.813, 175.107,
        113.919, 73.087,
        72.024, 47.622, 45.960, 43.852
    ])
    y_data = y_data_original * scaling_factor

    # Initial values: P0 = [rho1, rho2, theta0, dtheta,
    #                        s0, s1]
    P0 = np.array([6.59117, 0.40532, 543.122, 19.199,
                   0.04120, 0.08803])
    labels = ['rho1', 'rho2', 'theta0', 'dtheta', 's0',
              's1']

    # Parameter bounds for L-BFGS-B optimizer
    bounds = [
        (1, 10),          # rho1
        (0.01, 2),        # rho2
        (100, 600),       # theta0
        (1, 100),         # dtheta
        (0.001, 0.1),     # s0
        (0.001, 10)       # s1
    ]

#
=====
# 2. OBJECTIVE FUNCTION S DEFINITION (Negative
#   Log-Likelihood)
#
=====
def objective_s(P):
    """Calculates objective function S for the ELS
        framework."""
    rho1, rho2, theta0, dtheta, s0, s1 = P

```

```

# Boltzmann model
y_model = rho2 + (rho1 - rho2) / (1 +
    np.exp((x_data - theta0) / dtheta))

# Linear error model s_c = s0 + s1 * y_model
sigma = s0 + s1 * y_model

# Penalty for unphysical parameters
if np.any(sigma <= 1e-9) or dtheta <= 1e-9:
    return 1e15

# Objective function S
term1 = 2 * np.sum(np.log(sigma))
term2 = np.sum(((y_data - y_model) / sigma) ** 2)
return term1 + term2

#

```

```

# 3. OPTIMIZATION AND ERROR ANALYSIS
#

```

```

def run_els_fit():
    print("---- STARTING ELS MINIMIZATION ----")
    result = minimize(
        objective_s, P0, method='L-BFGS-B',
        bounds=bounds, options={'ftol ': 1e-9}
    )
    P_fit = result.x
    S_min = result.fun

    print(f"Status: {result.message}")
    print(f"Minimum S-value: {S_min:.6f}\n")

    # Hessian and covariance matrix
    H_num = nd.Hessian(objective_s)(P_fit)
    V_num = np.linalg.inv(H_num)
    std_err = np.sqrt(np.diag(V_num))
    R_matrix = V_num / np.outer(std_err, std_err)

    print("Optimized Parameters and Standard Errors:")

```

```

for l, val, err in zip(labels, P_fit, std_err):
    print(f"    {l:7s} = {val:10.5f} +/- {err:8.5f}")

print("\nCorrelation Matrix R:")
print(pd.DataFrame(R_matrix, index=labels,
    columns=labels).round(4))

return P_fit, std_err, R_matrix

#
=====
# 4. VISUALIZATION OF RESULTS (PLOTING)
#
=====
def plot_results(P_fit, filename: str =
    "els_fit_n404.png"):
    rho1, rho2, theta0, dtheta, s0, s1 = P_fit

    x_line = np.linspace(0, 670, 500)
    y_model_line = rho2 + (rho1 - rho2) / (1 +
        np.exp((x_line - theta0) / dtheta))
    sigma_line = s0 + s1 * y_model_line

    # Model values at data points
    y_model_pts = rho2 + (rho1 - rho2) / (1 +
        np.exp((x_data - theta0) / dtheta))
    residuals = y_data - y_model_pts

    fig, (ax1, ax2) = plt.subplots(
        2, 1, figsize=(9, 7), sharex=True,
        gridspec_kw={'height_ratios': [3, 1]}, dpi=150
    )

    # Main plot: Measurement data & fit
    ax1.scatter(x_data, y_data, color='#1f77b4',
        alpha=0.7, label='Measured Data N404', zorder=3)
    ax1.plot(x_line, y_model_line, color='black',
        lw=2, label='ELS Boltzmann Fit')
    ax1.fill_between(

```

```

        x_line , y_model_line - sigma_line ,
        y_model_line + sigma_line ,
        color='gray' , alpha=0.3, label=r'Uncertainty
        Band  $\pm 1 \sigma_{\mathrm{c}}$ '
    )
    ax1.set_ylabel(r'Dislocation Density  $\rho$ 
    [ $10^{14} \mathrm{m}^{-2}$ ]' )
    ax1.set_title('ELS Fit of Dislocation Density',
        fontsize=12, fontweight='bold')
    ax1.grid(True, linestyle='--', alpha=0.5)
    ax1.legend()

# Residual plot
ax2.axhline(0, color='black', linestyle='--', lw=1)
ax2.scatter(x_data, residuals, color='red',
    alpha=0.7)
ax2.set_xlabel(r'Temperature  $\vartheta$ 
    [ $^{\circ}\mathrm{C}$ ]' )
ax2.set_ylabel('Residuals')
ax2.grid(True, linestyle='--', alpha=0.5)

plt.tight_layout()
plt.savefig(filename)
plt.close()
print(f"\n[INFO] Plot saved as: {filename}")

#
=====
# EXECUTION
#
=====
if __name__ == "__main__":
    P_fit, std_err, R_matrix = run_els_fit()
    plot_results(P_fit)

```

7 Conclusion & Outro

Speaker Script & Visualization

[16:30–18:00] **Conclusion: Strengths of the Extended Least Squares (ELS) Method**

Camera: Close-up on host, key bullet points highlighted on screen.

Speaker: Let's summarize the key insights of today's episode!

The **Extended Least Squares (ELS)** method is an exceptionally powerful tool in scientific data analysis and engineering practice. Its primary advantage: It allows us to **calculate model parameters and uncertainty bounds simultaneously**.

Why should you use ELS in your projects?

1. **Broad Applicability:** ELS applies universally to optimization and regression problems well beyond materials science.
2. **Ideal for Small Data Sets:** It is especially valuable when working with limited measurement data, as it extracts maximum information from available data points.
3. **Robust Weighting:** It yields stable, reliable estimates even under large variations in weighting factors or measurement uncertainties.

Instead of treating uncertainties as a nuisance, ELS integrates them directly into the modeling process. You obtain mathematically sound, honest confidence intervals and prevent unphysical predictions!

Thank you for watching! If you found this episode helpful, hit like and subscribe to the channel. You can find the full Python code and data sets in the video description below. See you next time!